Deploy Doctor: Roadmap

Oct 29, 2026 · @Nandha

The problem and who it's for

Deploy Doctor tells a developer, in plain language, whether their deployed app is set up safely, and exactly how to fix what isn't on their host.

It's for someone who has just deployed their first or second app on a platform such as Render, Vercel or Railway. The app works, but they don't know what they don't know: they've never heard of CSP or `X-Forwarded-For`, and they won't read security documentation.

The idea comes from deploying Startly in October 2026. Every problem there was invisible until someone checked by hand:

- The rate limiter could be bypassed on Render, because the host's proxy chain wasn't what the documentation suggested.
- CORS blocked the new frontend until the backend was told its address.
- A misspelled environment variable was silently ignored instead of failing.

Deploy Doctor runs those checks automatically, so a beginner finds the problems before anyone else does.

What makes it different

Deploy Doctor checks a frontend and its API as a pair, tests how they actually behave, and explains every finding for beginners with a fix for their host. No existing tool combines those three.

Tool	What it does well	What it misses for a beginner
Mozilla Observatory, securityheaders.com	Grades security headers	Headers only; explanations written for experts
Lighthouse	Performance and accessibility	Not deployment or backend safety

Tool	What it does well	What it misses for a beginner
OWASP ZAP, Nuclei	Thorough security scanning	Overwhelming; built for security professionals
SSL Labs, testssl.sh	Deep HTTPS analysis	Far more detail than "is my HTTPS fine?"

The behaviour checks are the original part: does CORS really block other sites, can the rate limit be bypassed, how long does a cold start take. Header graders only read what a server claims; Deploy Doctor checks what it does.

Decisions so far

Version 1 is a Python command-line tool with gentle checks only, fix hints for Render first, and code written by you.

Topic	Decision	Why
Name	Deploy Doctor (`deploy-doctor`)	Says what it does; availability on PyPI and GitHub still to confirm
Form	Terminal only, no web frontend	Focus on backend skills first; a web version can reuse the JSON output later
Language	Python	The language you know from Startly's backend
Version 1 scope	Gentle checks only	Safe to run, quick to build; active checks come in a later phase behind a flag
Hosts	Render first, with a generic fallback	The host you know; Vercel and others later
Who writes the code	You write it; Claude teaches and reviews	So you can explain every line in an interview

Version 1 checks

Version 1 has six checks. Each sends a few ordinary requests and reports pass, warning or fail, with a plain explanation and a fix.

Check	What it does	Example finding
HTTPS and redirects	Requests the `http://` address and follows redirects	"Plain HTTP doesn't redirect to HTTPS"
Security headers	Reads CSP, HSTS, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`	"No Content Security Policy: here's what it blocks, and a starter policy"
CORS behaviour	Asks the API as your frontend, then as a made-up site	"Your API accepts requests from any website" or "it blocks your own frontend"
Health and latency	Calls a health path (default `/health`) a few times	"API answered in 120 ms" or "no health endpoint found"
Cold start	Compares the first response time with the next ones	"First request took 48 s: free tier waking up? Say so in your README"
Exposed files	Requests a short list of paths such as `/.env` and `/.git/config`	"`/.env` is publicly downloadable"

Every finding answers three questions: what's wrong, why it matters, and how to fix it on Render (or generally). The example command:

```
deploy-doctor https://startly-analyze.onrender.com --api https://startly-api.onrender.com
```

Startly is the first test patient. A deliberately broken demo app will show every check failing.

Responsible use

Deploy Doctor is for apps you own or have permission to test, and its design enforces that as far as a tool can. Against someone else's site, unusual requests can look like an attack and may be illegal.

Three levels of checks:

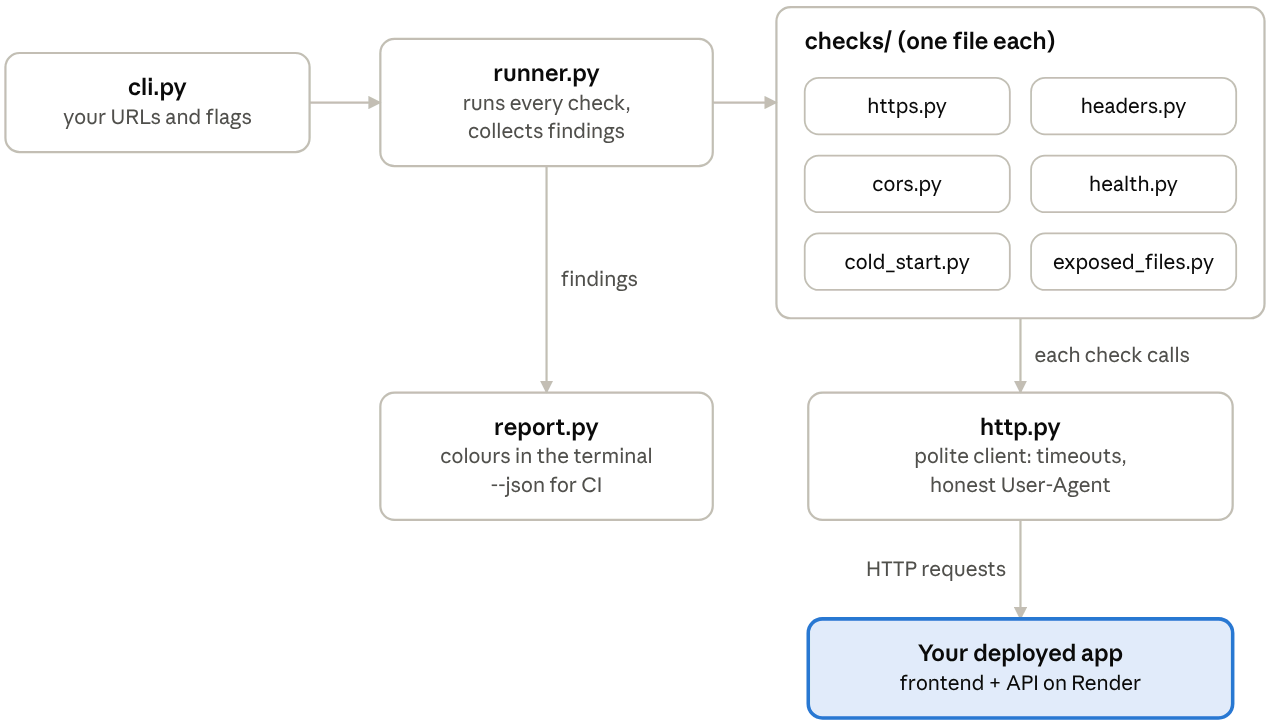
Level	Checks	How it runs
Gentle	HTTPS, headers, CORS, health, cold start	By default; ordinary requests a browser could send
Probing	Exposed files	Asks once "Do you own this site?" (or `--yes-i-own-it` in CI)
Active (later phase)	Rate-limit spoofing test	Only with `--active` plus the same confirmation

Rules built in from the start:

- Few requests, short timeouts, and a pause between bursts.
- An honest `User-Agent` (`DeployDoctor/<version>`), so site owners can see what called them.
- A clear line in the README and in `--help`: only test apps you own or have permission to test.

Architecture

Each check is its own small file that returns findings; everything else just runs the checks and prints the results.



Deploy Doctor architecture · 6 checks, 1 runner, 1 report

A finding is one small record: the check's name, a level (pass, warning, fail or info), what was found, why it matters, and how to fix it. Checks never print anything themselves. That keeps them easy to test with fake responses, and lets the same findings become coloured terminal output or JSON.

Project layout:

```
deploy-doctor/
├── pyproject.toml
├── src/deploy_doctor/
│   ├── cli.py
│   ├── runner.py
│   ├── http.py
│   ├── models.py        # the Finding record
│   ├── report.py
│   └── checks/
│       ├── https.py
│       ├── headers.py
│       ├── cors.py
│       ├── health.py
│       ├── cold_start.py
│       └── exposed_files.py
├── tests/
└── .github/workflows/ci.yml
```

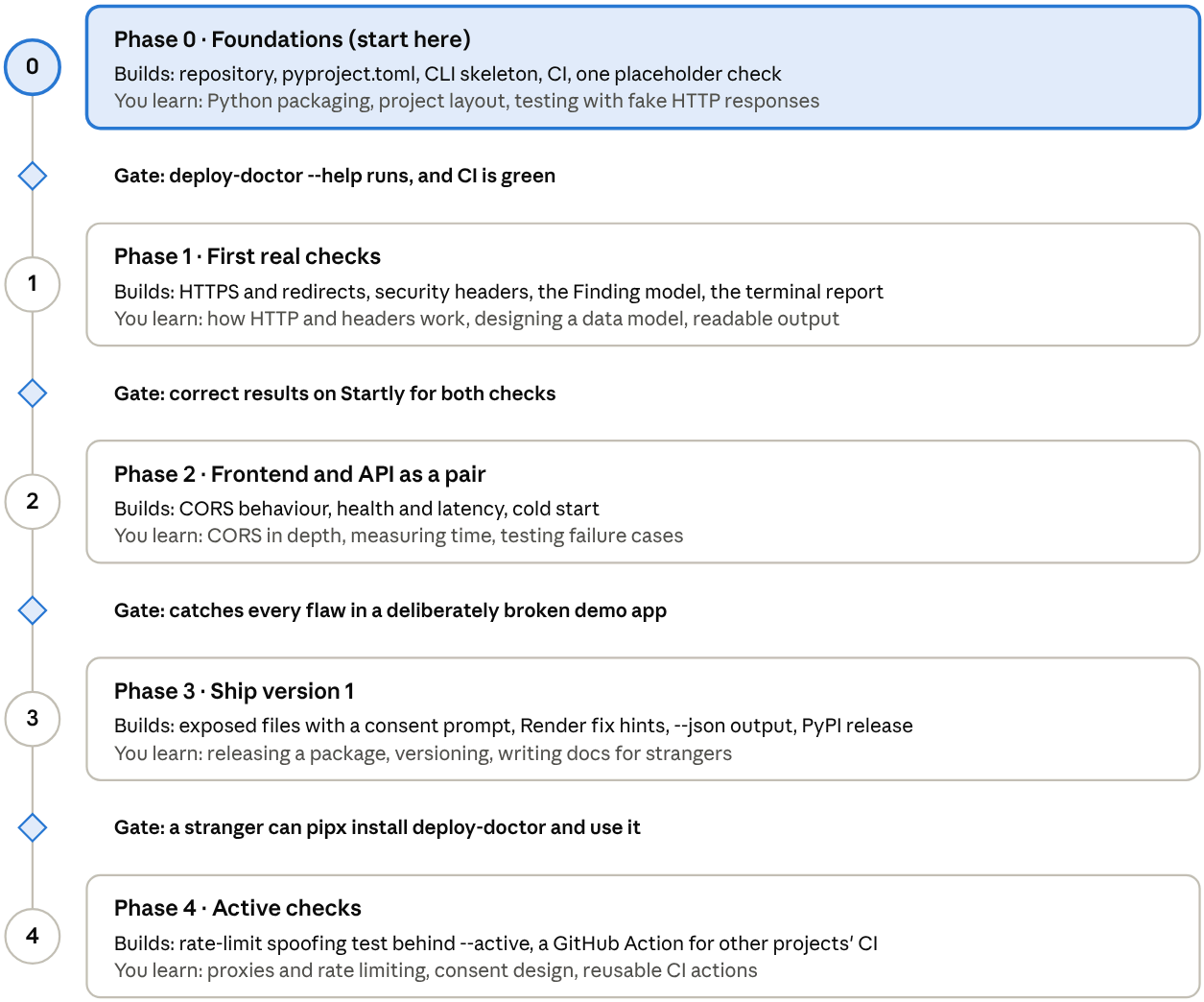
Libraries: `httpx` for requests (its built-in fake transport makes testing easy) and `rich` for the terminal output.

Phases

Version 1 ships at the end of phase 3; phase 4 adds the active checks. A phase is finished only when its gate passes.

Five phases, each ending at a gate you can check

No dates: each phase takes as long as the learning takes



Deploy Doctor phases · 5 phases, 4 gates

Phase 0 is mostly setup, but it decides how easy everything after it will be: with CI and a fake HTTP transport in place from the start, every later check can be written test-first.

How we work

You write the code; Claude teaches each new idea first and reviews your work after, the same way as the Docker and CI lessons.

Each piece of work follows one loop:

1. **Lesson:** a short explanation of the concept (for example, how CORS works), with no finished code.

2. **Homework:** you write the check and its tests.
3. **Review:** Claude reads the files and reviews them like a code review at work: must fix, should fix, good, each with the reason.
4. **Done** when the tests pass in CI, the check gives the right answer on Startly, and its explanation and fix read clearly to a beginner.

The project gets its own GitHub repository and CI from the first day, using what you learned on Startly.

Open questions and risks

The biggest risk is wrong answers: a check that reports a problem that isn't one, or misses a real one, loses a beginner's trust quickly.

Risk	Mitigation
False alarms, for example a CDN adds or strips headers	Test every check against Startly and the broken demo app; say "warning" when unsure, never "fail"
Hosts behave differently (Render sits behind Cloudflare)	Start with Render only; add hosts one at a time with real measurements
Cold-start detection is a guess	Report it as a hint with the measured times, not a verdict
Scope creep	Version 1 is exactly the six checks; new ideas go into a later-phase list
Someone points it at sites they don't own	Gentle defaults, confirmation for probing checks, a clear warning

Open questions:

- [ ] Is `deploy-doctor` free on PyPI and GitHub?
- [ ] Which command-line library: `argparse` (built in) or `typer` (friendlier)?
- [ ] Where should the broken demo app live: in this repo or its own?